Optimizing Breadth-First Search on the NVIDIA DGX Spark

Student: Fabio Missagia
Supervisors: Flavio Vella, Thomas Pasquali, Jacopo Raffi

Department of Information Engineering and Computer Science, University of Trento





NVIDIA DGX Spark: general properties

Property	Value
System	NVIDIA DGX Spark
CPU	20 cores: 10× Cortex-X925 + 10× Cortex-A725 (aarch64)
GPU	NVIDIA GB10 — compute capability 12.1 (`sm_121`)
Memory	128 GB unified LPDDR5x, CPU–GPU coherent, ∼273 GB/s *shared*
L2 cache	24 MiB
SMs	48
GPCs	4 (12 SMs each) — *measured, see next slides*
Max grid	2 147 483 647 × 65 535 × 65 535

One *physical* memory pool, hardware coherent: a kernel dereferences a plain `malloc()` pointer with no `cudaMemcpy`. ∼273 GB/s is an order of magnitude below datacenter HBM, like H100 (∼2.0 TB/s).

NVIDIA DGX Spark: per-SM properties

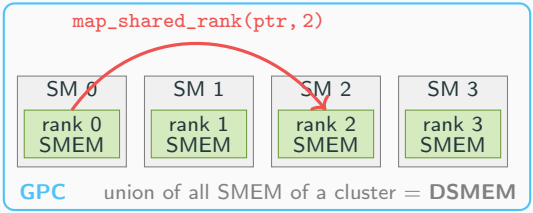
Property	Value
Max threads / block	1024
Max threads / SM	1536 (48 warps)
Max blocks / SM	24
Registers / SM	65 536
Unified L1 + shared memory	128 KiB
Max shared memory / SM	102 400 B (100 KiB)
Default shared memory / block	49 152 B (48 KiB)
Max shared memory / block (opt-in)	101 376 B (99 KiB)
Reserved shared memory / block	1 024 B

L1 and shared memory are the *same* 128 KiB SRAM, split by a configurable carveout: shared memory is bought at the cost of L1.

Thread block clusters (compute capability 9.0+)

An optional level between block and grid:

- **adjacent thread blocks** are grouped into clusters
- guaranteed to be **co-scheduled simultaneously inside one GPC**
- any thread block can address **any peer's shared memory** — loads, stores and *atomics* ride an **SM-to-SM** interconnect, not L2/DRAM ⇒ **DSMEM**
- **co-residency** is guaranteed
- hardware barrier across blocks: `cluster.sync()`



Thread block clusters on the DGX Spark

Measurement	portable	with opt-in
Cluster Size	8	12

- **8** is the maximum CUDA guarantees *portably* on any cluster-capable GPU — code using it runs anywhere.
- **12** is *this hardware's* real maximum, unlocked per kernel with `cudaFuncAttributeNonPortableClusterSizeAllowed` — at the price of code that is no longer portable. Above 12: *cluster misconfiguration*.
- `cudaOccupancyMaxPotentialClusterSize` is **per kernel**, not a device constant: the same kernel answers 8 before the opt-in and 12 after.
- 12 is *usable*, not merely launchable: DSMEM verified across all 12 ranks with a ring test (rank r reads rank $r{+}1 \bmod n$).

How many GPCs?

Problem. No CUDA attribute exposes the GPC count or the SM-to-GPC mapping.

Idea. Exploit the one guarantee available: *all blocks of a cluster are co-scheduled in a single GPC*. So the SMs inside one cluster are provably all in the *same* GPC.

Method. Launch of 4 clusters $\times$ 12 blocks.

- every block reads the `%smid` register and reports the **SM it ran on** and the **cluster it belongs to**
- **52 KB of shared memory per block** forces one block per SM, so a 12-block cluster covers 12 *distinct* SMs
- cluster size 12 = one GPC's SM count, so **4 clusters cover all 48 SMs** exactly once and each cluster is a GPC
- **tried many trials**, but the result is always the same and all 48 SMs are observed every time, so one launch is enough

GPC topology: 4 GPCs × 12 SMs

Example:

GPC	SM ids											
GPC 0	0	1	8	9	16	17	24	25	32	33	40	41
GPC 1	2	3	10	11	18	19	26	27	34	35	42	43
GPC 2	4	5	12	13	20	21	28	29	36	37	44	45
GPC 3	6	7	14	15	22	23	30	31	38	39	46	47

Derived and verified:

- `TPC = smid/2,  GPC = (smid/2) % 4`
- 24 TPCs of 2 SMs, 6 TPCs per GPC, round-robin across GPCs

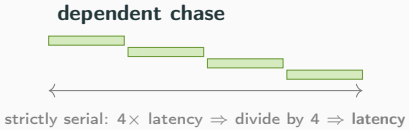
Three independent cross-checks:

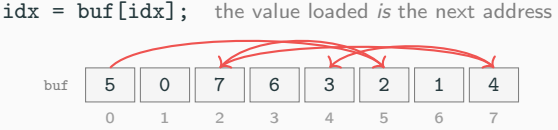
- 4 clusters of 12 *distinct* SMs, 48 SMs observed, none twice
- $4 \times 12 = 48$ matches `multiProcessorCount`
- the largest launchable cluster (12) *equals* one GPC's SM count

How do we measure latency?

A GPU is built to hide latency: a load does not stall the thread, this is called Memory-Level Parallelism.

The fix: make each load depend on the result of the previous one, so that the thread cannot issue the next load until the previous one has completed. This is called a **dependent pointer chase**, or pChase.





In addition:

- **Sattolo cycle:** `buf` is one random cycle over *every* element: no short loops, nothing to prefetch.
- **steps:** how many dependent loads the timed loop performs. If far below the cycle length: no address is revisited.
- **reps:** The whole kernel is launched many times — a drift across reps exposes a cold cache.
- **Warm-up:** L1: one untimed lap to warm the cache; L2: a coalesced sweep *after* the kernel is loaded (lazy load flushes L2).

Latency Benchmark – Random pChase

Dependent random pointer chase, 16 384 steps, one thread, median of 101 reps.

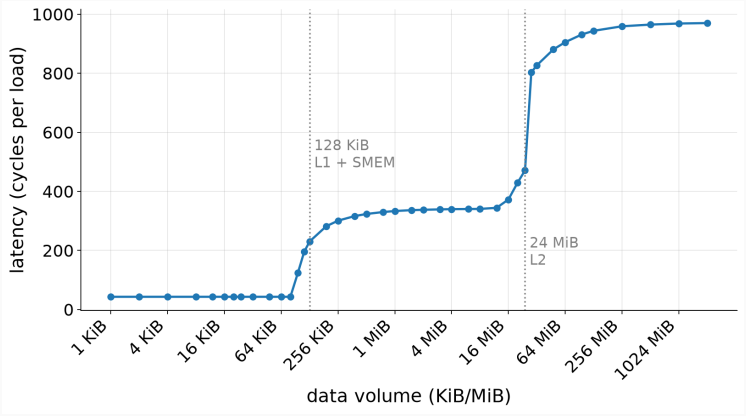
Placement	GB10 (cy)	ns	H800 (cy)*
L1 Cache	43.2	17.9	32.0
SMEM	32.9	13.7	29.0
SMEM, via DSMEM API	36.9	15.3	33.0
DSMEM	180-211	75-88	181-213
L2 Cache	340.1	141.7	264–502
DRAM	965.8	401.6	656

* Luo et al., arXiv:2501.12084 — Hopper H800, same p-chase method

Why does SMEM via DSMEM cost 4 more cycles? `map_shared_rank` returns a generic pointer, so the load compiles to `LD` (address space resolved by hardware on every access) instead of `LDS` (a plain shared-memory load).

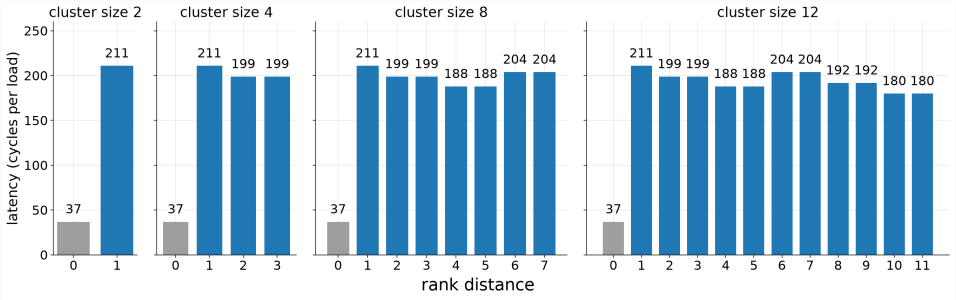
Latency Benchmark – Random pChase

Dependent random pointer chase, 16 384 steps, one thread, median of 101 reps.



Latency Benchmark – Random pChase

Same chase, but thread 0 of rank 0 accesses a *peer* block's DSMEM at rank distance d.



- **A genuine SM-to-SM path.** A remote access costs about *half* an L2 hit and a *fifth* of DRAM.
- **Flat across the cluster.** 210.9 cy at distance 1, identically at cluster sizes 2, 4, 8 and 12. Not monotone in distance: ranks pair up $\{2,3\},\{4,5\},\ldots$ — the two SMs of one TPC.
- **Independently confirmed.** H800 agrees within a few cycles; since GB10's L2 and DRAM are *slower* than Hopper's while its fabric is not, DSMEM's advantage is *larger* here.

Distance does not matter

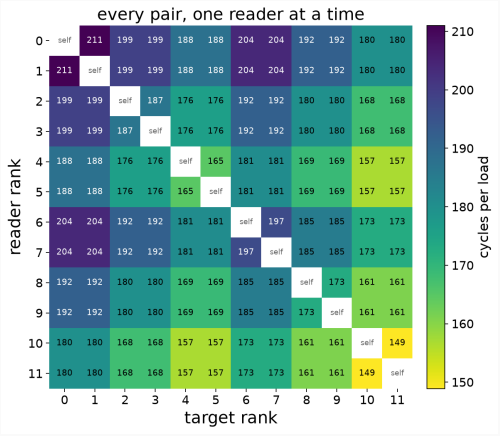
Rank 0 → its own TPC sibling (rank 1) is 210.9 cy, the *slowest* pair of all...

Rank 0 → the far rank 10 is 179.9 cy, the *fastest* pair of all...

Let's measure every pair!

Facts from the matrix

- perfectly **symmetric**
- TPC siblings have **identical rows**
- the diagonal is **not flat** (211 vs 149)
- **same results** in all 4 GPCs



If distance does not matter, what does?

Over all 36 TPC pairs the following holds with zero error:

$$\text{latency}(k \to d) = 149 + w_k + w_d$$

TPC	5	2	4	1	3	0
w **(cy)**	0	8	12	19	24	31

Maybe a **shared meeting point**?

Every request may go SM → hub → SM.

Independent confirmation: reading *L2* from every SM shows the same pattern!

median L2 latency (cy), per TPC

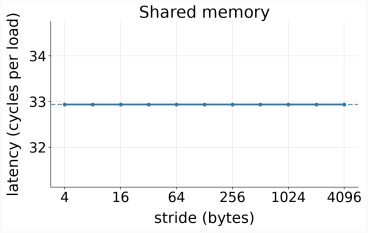
TPC pos.	0	1	2	3	4	5
GPC 0	343	331	320	336	324	312
GPC 1	318	306	295	311	299	287
GPC 2	318	306	295	311	299	287
GPC 3	341	329	318	334	322	310

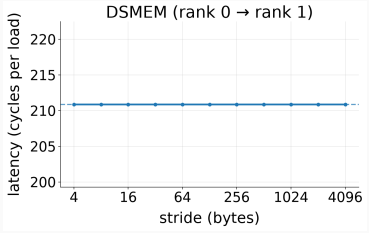
row minus its minimum = per-TPC distance

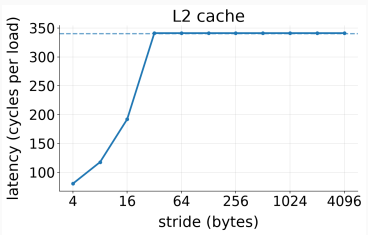
TPC pos.	0	1	2	3	4	5
GPC 0	**31**	**19**	**8**	**24**	**12**	**0**
GPC 1	**31**	**19**	**8**	**24**	**12**	**0**
GPC 2	**31**	**19**	**8**	**24**	**12**	**0**
GPC 3	**31**	**19**	**8**	**24**	**12**	**0**

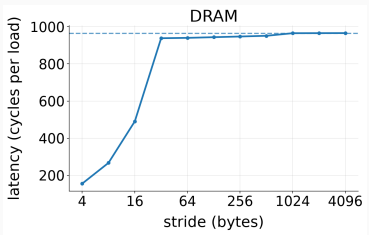
Latency Benchmark – Linear pChase

Same chase, but element i points to $i + s$, $s = 4\,\mathrm{B} \ldots 4\,\mathrm{KiB}$. Dashed = random chase (previous slide).









- **SMEM/DSMEM are pattern-blind**
- **L2/DRAM see only the 32 B sector** $k = 32/s$ loads share one L1 fill, $(T + (k-1) \cdot 43.2)/k$ gives 80/118/192 cy (L2) and 155/266/490 cy (DRAM) — as measured.
- **No hardware prefetcher:** from 32 B on, linear = random.

Clusters are not free

The moment a launch carries a cluster dimension, the device holds **2 blocks per SM at most (96 in total), whatever the block size.**

blockDim	total blocks (unclustered)	total blocks (clustered)	penalty	occupancy (clustered)
64	1152	96	12×	8%
128	576	96	6×	17%
256	288	96	3×	33%
512	144	96	1.5×	67%
1024	48	48	none	67%

- **Confirmed two independent ways:** `cudaOccupancyMaxActiveClusters`, and a test in which every block must observe all the others before a timeout.
- **Didn't found anything about this on the web...**

Clusters are not free

The cap also moves with the *shared memory* a block requests. Past ∼50 KiB only one block fits per SM, so the device holds 48.

SMEM / block	cs = 1	cs = 2	cs = 4	cs = 8	cs = 12
0	96	96	96	96	96
32 KiB	96	96	96	96	96
50 KiB	48	48	48	**32**	48
99 KiB	48	48	48	**32**	48

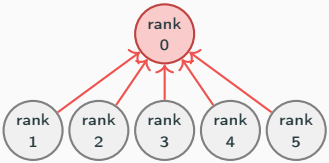
total resident blocks device-wide (48 SMs), cs = cluster size

- A cluster never spans a GPC, and a GPC has 12 SMs. One cluster of 8 leaves 4 SMs free — too few for a second — so each of the 4 GPCs runs one cluster and strands 4 SMs: $4 \times 8 = 32$, a third of the GPU idle.
- **12 and 6 tile the machine exactly**; 8, the *portable* maximum, does not.

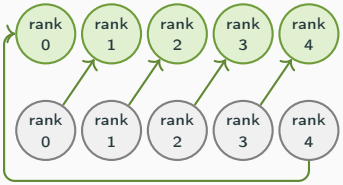
Throughput Benchmark

So far: *one* thread reading *one* peer. Now: *many* threads, reading the *same* peer or *different* peers.

all vs 1 (`broadcast`)



all vs all (`ring`)

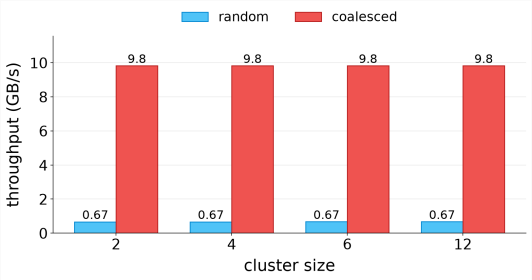


- Two access patterns:
  - `random`: each lane starts from a different random position in the same cycle, resulting in approximately 32 load chains per warp.
  - `coalesced`: lane l starts at $base + l \Rightarrow$ a warp reads 32 consecutive words, one 128 B line, a *single* request.

Throughput Benchmark – Broadcast

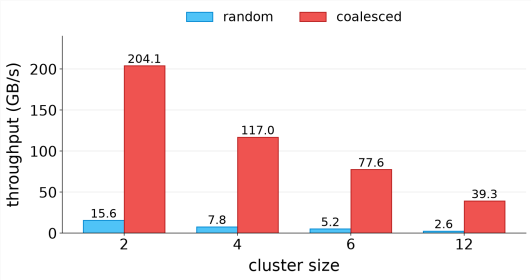
In every cluster, every thread of ranks $1 \ldots N-1$ reads **rank 0**'s shared memory.
1 block per SM, 256 threads per block.

One cluster



1 cluster of N blocks: only N SMs busy.

Whole GPU

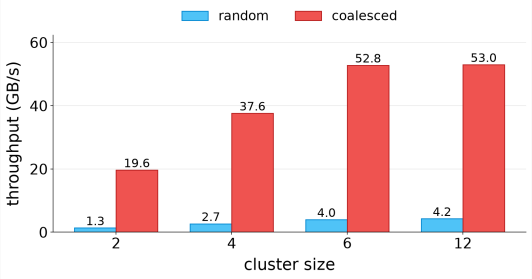


48/N clusters of N blocks side by side: all 48 SMs busy.

Throughput Benchmark – Ring

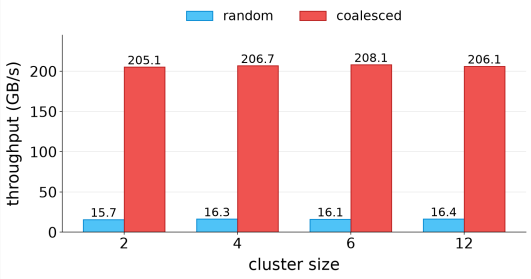
In every cluster, every thread of rank k reads **rank** $k+1 \pmod N$: one reader per owner.
1 block per SM, 256 threads per block.

One cluster



1 cluster of N blocks: only N SMs busy.

Whole GPU



48/N clusters of N blocks side by side: all 48 SMs busy.

Throughput Benchmark – Summary

scope	**coalesced**	random	what sets the limit
one owner SM	**9.8 GB/s**	0.67 GB/s	the owner: 128 B every ∼31 cy ≈ 4.1 B/cy
one GPC (12 SMs)	∼**53 GB/s**	4.3 GB/s	the GPC's SM-to-SM network
whole chip (48 SMs)	∼**206 GB/s**	16 GB/s	4 GPCs, independent

- **coalesced** is limited by *bytes*, **random** by *requests* (one every ∼14 cy per owner, only 4 B each): 15× apart.
- The owner here is SM 0, the worst position in the GPC; the best-placed owner reaches ∼17 GB/s (∼7 B/cy).
- The GPC network saturates with ∼6 SMs reading: beyond that, adding SMs only splits the same ∼53 GB/s.

What's next?

1. **Characterization of DSMEM**
   - Performance characterization and impact of DSMEM
   - Limitations and assumptions of the DSMEM programming paradigm
   - Theoretical upper bounds on performance improvements
   - Applicability: algorithms that can and cannot exploit DSMEM
2. **State-of-the-art GPU-accelerated BFS**
   - Microbenchmark access pattern of a BFS?
   - Graph partitioning strategies
   - Work-stealing techniques
   - Data types and data structures for high-performance BFS
3. **BFS implementation based on DSMEM**
   - Algorithm design informed by (1) and (2)
   - Graph partitioning (probably 2D) and work stealing
   - Integration of *mxint* (micro-scaling integer type), if worthwhile
   - Performance evaluation; limitations and future directions